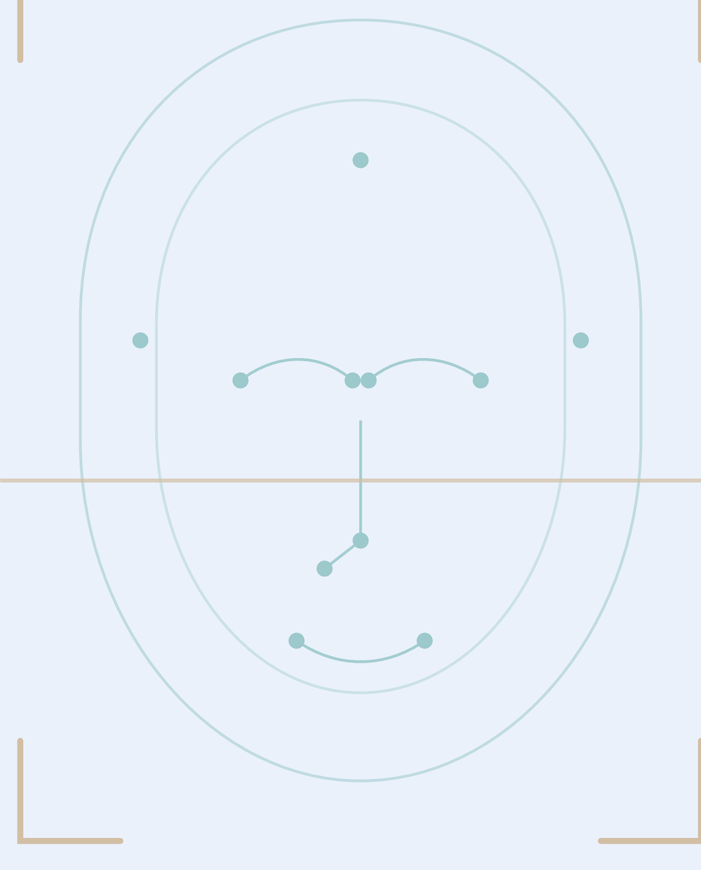
UNIVERSIDAD ANDINA DEL CUSCO
Facultad de Ingeniería y Arquitectura · Escuela Profesional de Ingeniería de Sistemas

SPRINT 3 · CRISP-DM · FASE 6: DESPLIEGUE



Autenticación biométrica por reconocimiento facial

Del modelo evaluado a una aplicación web funcional: Flask + FaceNet (VGGFace2) + MTCNN, con el umbral del Sprint 2 sin modificar.

EQUIPO	ASIGNATURA	DOCENTE
TRALALEROS	**Inteligencia Artificial**	**Ing. Espetia Huamanga Hugo**

Andia Sullcaccori, Jose Adelky · Giraldo Altamirano, Rodrigo Eduardo · Khuno Mamani, Cristian Maycol · Quispe Rivera, Josue · Cusco · Perú · 2026

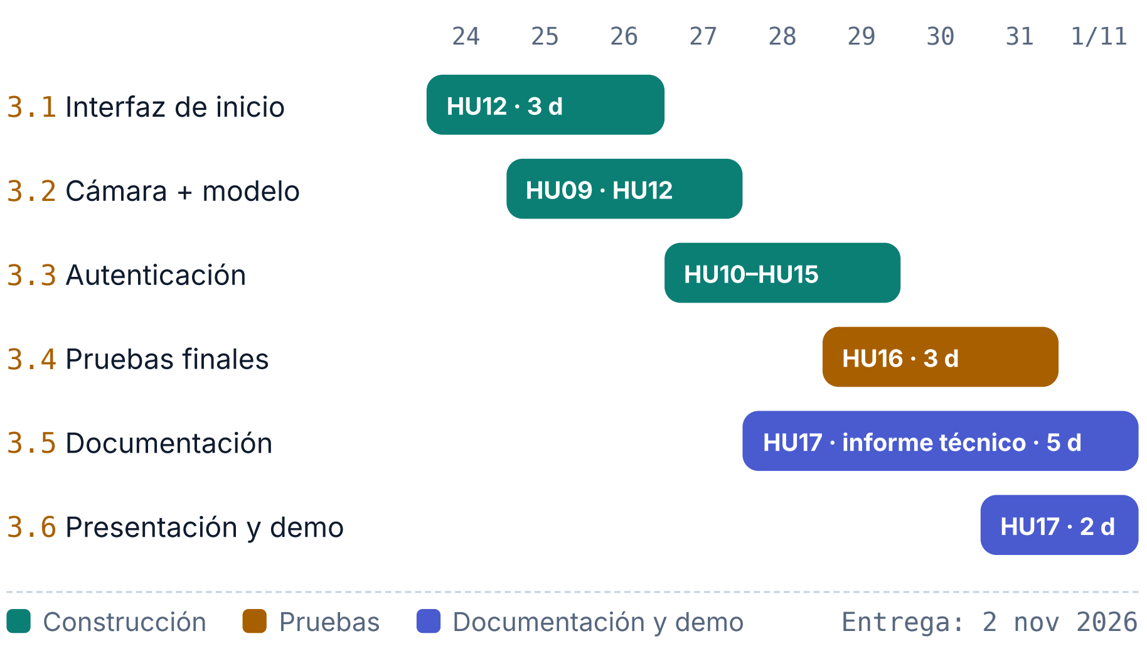
134/134	**0,268** s	**0,5514**	**3** de 5	**0,041**%	**1,00**%
pruebas automáticas pasan (pytest)	mediana por intento en servidor (GPU)	umbral de similitud coseno (Sprint 2)	fotogramas deben superar el umbral	FAR · 6 de 14 725 impostores	FRR · 3 de 299 genuinas

01 · RECOMENDACIONES DEL DOCENTE

Lo que se pidió y cómo se cumplió

1. **Fases de CRISP-DM sucesivas**
   Solo la fase 6. Modelo, plantillas y umbral del Sprint 2 sin cambios, verificados por SHA-256.
2. **Software separado de la ciencia de datos**
   La app vive en `app/` y usa el modelo como biblioteca: `src.pipeline.Verifier`.
3. **Tener la aplicación «subida»**
   Modo público por túnel de Cloudflare; la administración corre en otra instancia no expuesta.

CRONOGRAMA PLANIFICADO · 24 oct → 1 nov 2026 · 23 commits

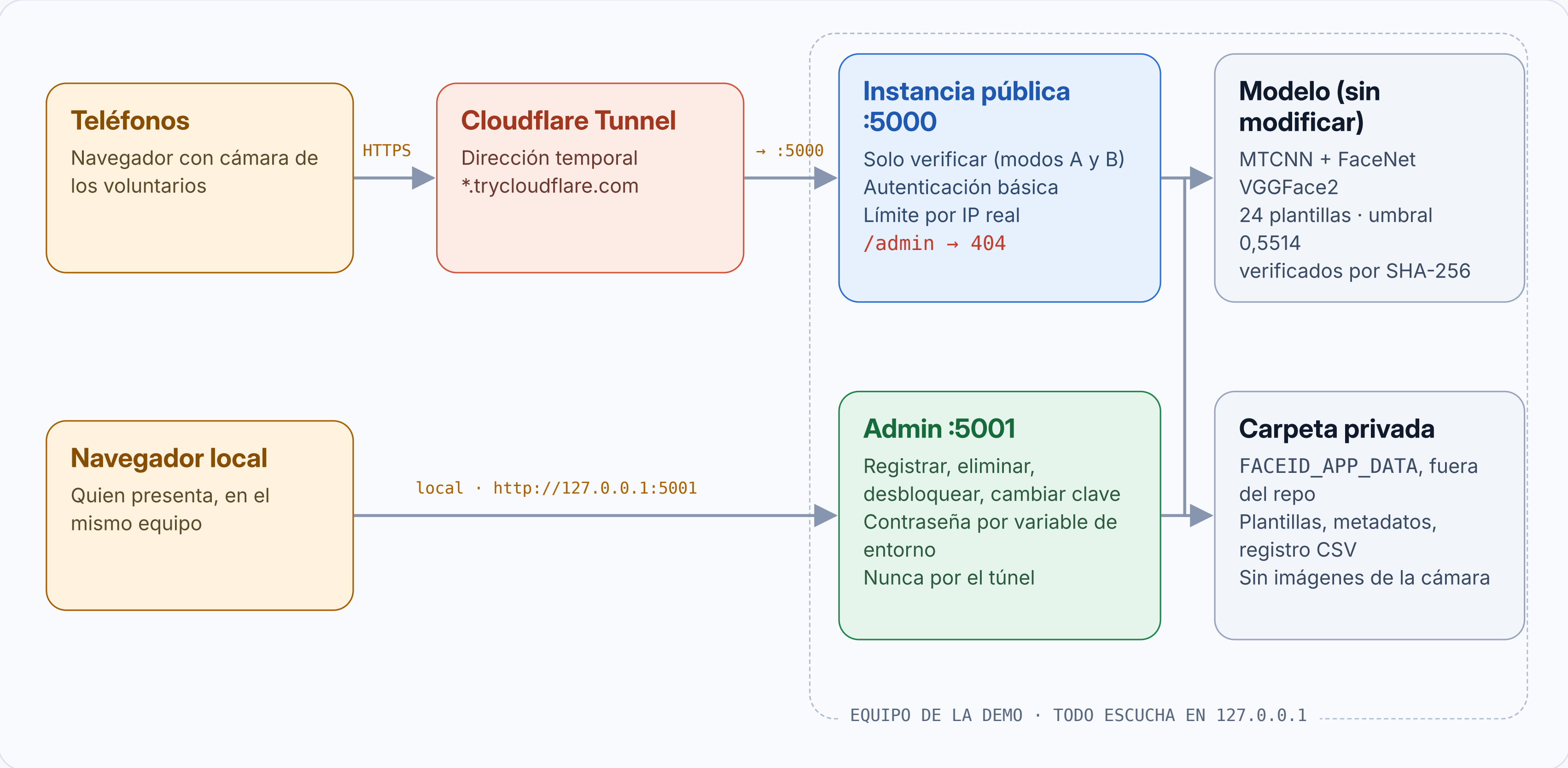


Entrega: 2 nov 2026

02 · ARQUITECTURA DEL DESPLIEGUE

Dos instancias, un modelo congelado

La pública solo verifica; la de administración solo se abre desde el propio equipo. Comparten la carpeta privada: un cambio en una aparece en la otra sin reiniciar.



Teléfonos
Navegador con cámara de los voluntarios

Cloudflare Tunnel
Dirección temporal *.trycloudflare.com

Instancia pública :5000
Solo verificar (modos A y B)
Autenticación básica
Límite por IP real
`/admin → 404`

Modelo (sin modificar)
MTCNN + FaceNet VGGFace2
24 plantillas · umbral 0,5514
verificados por SHA-256

Navegador local
Quien presenta, en el mismo equipo

Admin :5001
Registrar, eliminar, desbloquear, cambiar clave
Contraseña por variable de entorno
Nunca por el túnel

Carpeta privada
FACEID_APP_DATA, fuera del repo
Plantillas, metadatos, registro CSV
Sin imágenes de la cámara

EQUIPO DE LA DEMO · TODO ESCUCHA EN 127.0.0.1

Python 3.10.4 · Flask 3.1.3 · PyTorch 2.11.0 + CUDA 12.8 · facenet-pytorch 2.6.0 · OpenCV 5.0.0 · pytest 9.2.0
GPU RTX 5060 8 GB · CPU Core i5-10400 · 16 GB · SO Windows 10 Pro

03 · CÓMO DECIDE LA APLICACIÓN

De la cámara al veredicto en 6 pasos

01 Elegir el modo
A: nombre + rostro (1:1)
B: solo cámara (1:N)

02 Captura
5 fotogramas cada 400 ms (≈1,6 s), 640×480, enviados como JPEG

03 Detección y calidad
MTCNN recorta el rostro más grande a 160×160 y aplica 6 filtros

04 Vector de 512
FaceNet VGGFace2 congelada, solo con fotogramas válidos

05 Similitud coseno
A: con la plantilla elegida
B: con todos los usuarios de la app

06 Votación
≥ 3 de 5 fotogramas ≥ 0,5514
B: margen ≥ 0,05 sobre el 2.º

Acceso permitido
Reinicia el contador. Si el usuario tiene palabra clave, pasa al paso 2.

Acceso denegado
3 seguidos bloquean 60 s: al usuario (modo A) o a la IP (modo B).

No se pudo verificar
Menos de 3 fotogramas válidos. Muestra el motivo y no cuenta para el bloqueo.

Paso 2: palabra clave
Opcional. Se pide aunque el rostro falle, para no revelar qué factor falló.

04 · TRES FORMAS DE INGRESAR

Verificar, identificar y reforzar

A · EVALUADO EN SPRINT 2

Nombre + rostro · 1 a 1

El usuario elige quién es; su rostro se compara solo con esa plantilla.

99,48% exactitud balanceada · **0,67%** EER

B · NO EVALUADO

Solo cámara · 1:N

Compara con todos los usuarios de la app (nunca con las 24 identidades del dataset). Gana quien obtenga ≥ 3 de 5 votos y saluda «Hola, <seudónimo>».

Margen sobre el 2.º candidato: **0,05** (no calibrado, configurable con `FACEID_MARGEN_1N`)

OPCIONAL · A Y B

Palabra clave · segundo factor

Mínimo 6 caracteres. Guardada solo como hash `scrypt` con sal ($n = 2^{14}$, $r = 8$, $p = 1$), comparada en tiempo constante.

Token de un solo uso, ligado a la IP, caduca en **120 s** y vive solo en memoria.

05 · FILTROS DE CALIDAD POR FOTOGRAMA

Si no es confiable, lo dice

`sin_rostro` **Probabilidad < 0,90** No se detectó ningún rostro	`varios_rostros` **Otro ≥ 0,95 y ≥ 25 % del área** Hay más de un rostro
`lejos` **Lado menor < 60 px** El rostro está demasiado lejos	`oscuro` **Media de gris < 40** La imagen está muy oscura
`sobreexpuesto` **Media de gris > 225** Demasiada luz	`borroso` **Varianza laplaciano < 10** La imagen está muy borrosa

Calibrados con recortes de validación del dataset, aún no con la cámara.

06 · CONTROL DE INTENTOS Y SEGURIDAD

Defensas por capas

× 1 · × 2 · × 3 → 60 s

3 denegados seguidos → bloqueo de 60 s (por usuario en A, por IP con hash y sal en B). Los no concluyentes no suman.

10 / 300 s tope global por usuario, pausa de 30 s

20 / min verificaciones por IP en el túnel

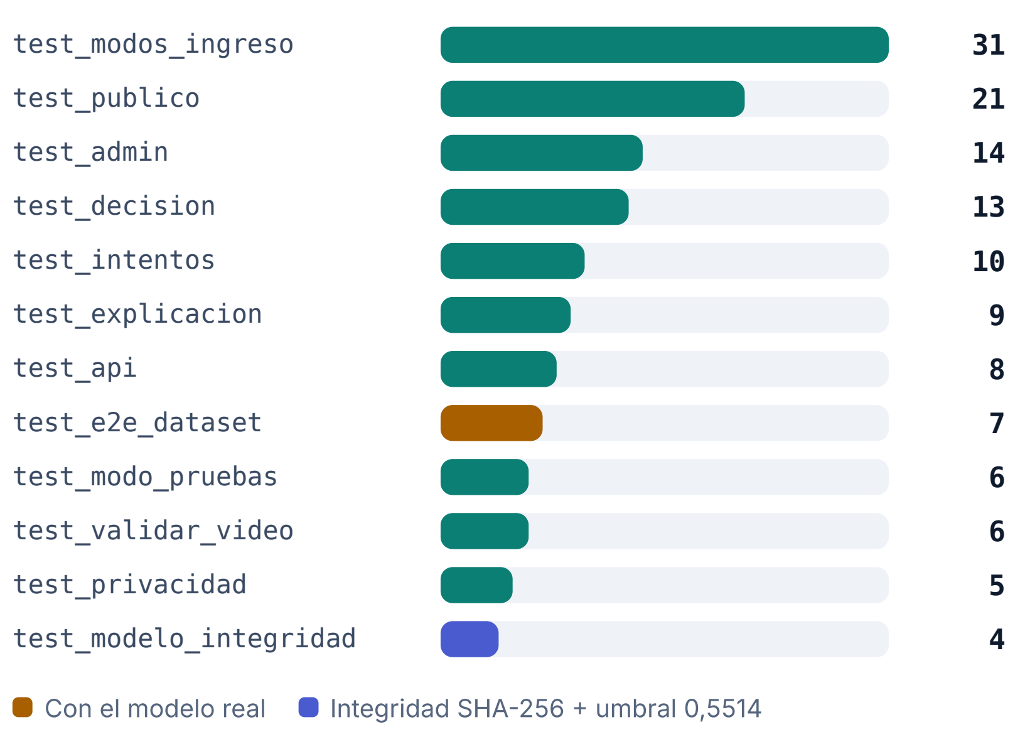
10 / 10 min contraseñas fallidas por IP (acceso público)

0 imágenes en disco: se procesan en memoria y se descartan

Registro con autorización escrita firmada · 10–20 fotos, ≥ 8 válidas · Plantilla = media renormalizada · Estado compartido con escritura atómica

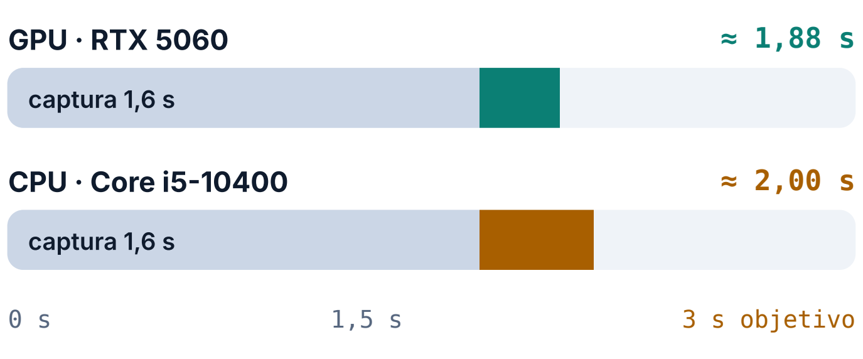
07 · PRUEBAS AUTOMÁTICAS

134 pruebas · **100 % pasan** · 29/10/2026



08 · TIEMPO POR INTENTO

Captura en el navegador + petición completa en el servidor, frente al objetivo de **≤ 3 s**.



0,268 s servidor GPU · p95 0,294

0,388 s servidor CPU · p95 0,402

Ensayo con 5 fotos del dataset y 20 repeticiones. Incluye decodificación, MTCNN, FaceNet y decisión.

JIT desactivado solo en la app: TorchScript recompilaba la red (13–23 s) cada vez que cambiaba el tamaño del lote. Los vectores son idénticos (diferencia 0).

09 · ENSAYO DE PUNTA A PUNTA

Con fotos del dataset

Identidad registrada, 5 fotos suyas	Permitido
Misma persona eligiendo otra identidad	Denegado
Cámara tapada	Sin rostro
Dos personas en el cuadro	Varios rostros
Imagen muy oscura	No concluyente
Registro nuevo (12 fotos) y verificación	Correcto
Modo B con 2 usuarios registrados	Identifica bien

Ensayo técnico, no uso real ni nueva evaluación del modelo.

10 · CASOS FUNCIONALES CON CÁMARA REAL

21 casos de prueba · CF01–CF21 21/21

CF01 Usuario registrado	CF02 Con lentes de ver	CF03 Lejos de la cámara
CF04 Muy cerca	CF05 Poca luz	CF06 A contraluz
CF07 Elige a un compañero	CF08 No registrado (modo A)	CF09 Cámara tapada
CF10 Dos personas	CF11 3 denegados → bloqueo	CF12 Desbloqueo por admin
CF13 Eliminar usuario	CF14 Foto/video (sin vida)	CF15 Sin autorización escrita
CF16 Modo B: registrado	CF17 Modo B: no registrado	CF18 Clave correcta
CF19 Clave incorrecta ×3	CF20 Modo B: bloqueo por IP	CF21 Modo B + clave

Todos con resultado «Positivo» (obtenido = esperado). CF14 confirma la limitación conocida: una foto o video del usuario puede ser aceptado porque no hay detección de vida.

11 · CUMPLIMIENTO DE REQUERIMIENTOS

13 de 13 atendidos

● Implementado ● Con matiz

RF01	RF02	RF03	RF04	RF05	RF06	RF07
Registro con autorización	Captura con la cámara	Detecta y extrae (ensayo)	Compara por similitud (ensayo)	Permite si cumple umbral	Deniega si no cumple	Mensaje comprensible

RNF01	RNF02	RNF03	RNF04	RNF05	RNF06
Interfaz sencilla: una pantalla, tres pasos	Tiempo adecuado: ensayo bajo 3 s	Datos restringidos, aún sin cifrado	Métricas en prueba independiente (Sprint 2)	Control de intentos no autorizados	Avisa si no puede verificar

12 · LIMITACIONES HONESTAS

Prototipo académico

Sin detección de vida
No distingue a una persona real de una foto, pantalla o video. La app lo advierte en sus páginas.

Métricas con fotografías
Dataset de Kaggle, sobre todo famosos; no cámara web.

Modo 1:N no evaluado
El riesgo crece con más usuarios; margen sin calibrar.

Bloqueos que afectan a otros
Cualquiera puede bloquear a un usuario fallando 3 veces.

Datos sin cifrar
Una sola contraseña de administrador.

Túnel temporal
La URL cambia en cada reinicio; el equipo debe seguir encendido.

Pocos voluntarios
Ensayos con dataset no reemplazan el uso real.

13 · CONCLUSIONES Y SIGUIENTES PASOS

Fase 6 completa en software

La aplicación usa el modelo y el umbral del Sprint 2 **sin modificarlos**, ofrece tres formas de ingresar, informa cuando no puede verificar, controla los intentos y se publica por túnel con la administración separada.

Pendiente del equipo

- → Medir tiempos con la cámara real y desde un teléfono
- → Probar el túnel con cloudflared y ajustar los filtros de calidad
- → Ejecutar una sola vez la prueba con video (tarea 2.6)

Antes de cualquier uso real

Detección de vida · Cifrado biométrico · Bloqueo no abusable · Evaluar 1:N con más personas

CÓDIGO FUENTE DEL PROYECTO

github.com/CryWar911/ReconocerRostro.git

README · limitaciones.md · guion_demo.md · casos_de_prueba.csv

TRALALEROS
Universidad Andina del Cusco · 2026
Referencias: Schroff 2015 · Cao 2018 · Zhang 2016 · Chapman 2000